Your First ZipperGen Workflow

A short path from a prompt to a prepared deployment

The ZipperGen Authors

Tutorial edition: 26 July 2026

Abstract

This tutorial builds one small multi-participant application from beginning to end. You will describe it in ordinary language, ask a coding assistant to produce visible Python, inspect and validate the resulting protocol, run it with a free mock model, interrupt and resume it, review it, and prepare a named deployment. Optional configuration and complete command explanations are kept in the companion manual so that the first successful cycle stays short.

What you need

Use macOS or Linux, a terminal, Git, and the `uv` Python tool. Codex or Claude Code is needed for prompt-to-code generation. If neither is installed, the repository contains a complete fallback workflow so that inspection, durable execution, and deployment can still be tried without an API key.

What you are about to build

The example accepts a request, lets a Writer draft an answer, asks a Reviewer to assess every draft, and leaves the final decision to a human. A rejected draft returns to the Writer while a bounded retry budget remains. An unapproved draft must never be returned.

This is a protocol, not merely a list of agents. One global Python workflow states who owns each value, who communicates with whom, and who owns each decision. ZipperGen projects that global workflow into an exact local program for every participant. Validation checks the global protocol and all of those projections before execution.

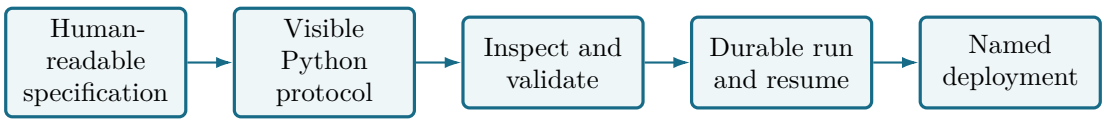

Figure 1: The first-workflow path. A coding assistant may propose the Python, but inspection, validation, human review, and deployment remain explicit.

The safety boundary

A coding-assistant process returning does not mean that its requested checks passed. Studio records those facts separately. Generated code is always reviewed before it is accepted, run, or deployed; opaque model output is never treated as an automatically deployed workflow.

1 Create the project and install ZipperGen

The tutorial project is the outer directory. The ZipperGen framework checkout lives below it:

```
mkdir -p "$HOME/Projects"
cd "$HOME/Projects"
mkdir zippergen-tutorial
cd "$HOME/Projects/zippergen-tutorial"
git init
git clone https://github.com/zippergen-io/zippergen.git zippergen
uv python install 3.12
uv sync --project zippergen --python 3.12
uv tool install --force --editable ./zippergen
```

This layout gives the application its own Git history and lets a repository-aware assistant work from the application root. The editable tool still uses the nested framework checkout. If `uv` is not installed, follow https://docs.astral.sh/uv/getting-started/installation/, restart the terminal if requested, and repeat the commands beginning with `uv python install`.

2 Open Studio and initialize the project

Remain in the outer project directory:

```
cd "$HOME/Projects/zippergen-tutorial"
zippergen
```

The beginning reports the project root, whether Codex or Claude is available, and one suggested next command. At the Studio prompt, enter:

```
project init zippergen-tutorial
```

Studio creates `zippergen.toml` and safe Git ignores. It stores machine-specific runs and secrets outside the repository, so this tutorial does not require a `ZIPPERGEN_HOME` export or a hand-written SQLite path.

From now on, commands such as `workflow create`, `run`, and `deploy` are Studio commands. Do not prefix them with `zippergen`. Press Tab for completion or enter `help` if the next step is forgotten.

3 Write the workflow specification

Enter:

```
workflow create
```

Studio creates the fixed file `specification.md` and opens it in an automatically discovered terminal editor. You do not choose a prompt filename. Replace the short writing guide with the following specification, save, and exit the editor:

```
# Reviewed Answer

Produce a helpful answer to a request, subject to automated review and explicit
human approval.
```

```
## Participants and behavior

- Requester initially owns `request` and `max_retries`.
- Writer uses an LLM to draft and revise an answer.
- Reviewer uses a separate LLM to assess every draft.
- HumanApprover owns the decision to approve or reject a draft after the
  automated assessment.
- A rejection sends the draft back to Writer while retries remain.
- Never return an unapproved draft. Return an explicit failure after retry
  exhaustion.
- Declare an optional logical Telegram connector named `human-approval` for
  HumanApprover, with notification and approval capabilities. Terminal
  approval must remain available when it is unbound.

## Inputs and operation

- The application accepts a `request` input, defaulting to
  `Explain why the sky is blue in two sentences.`
- It accepts an integer `max_retries` input, defaulting to `2`.
- Use the deterministic mock model by default.
- A deployment may instead use OpenAI or Anthropic. Request the corresponding
  API key only when that provider is selected.
```

The durable design intent now lives in one ordinary, versionable file. Development mechanics such as filenames, tests, validation commands, and the prohibition on automatic deployment are added by Studio to the generated assistant task; they do not have to be copied into the specification.

4 Produce visible Python

The Studio welcome message says whether a supported assistant is available. Choose exactly one:

```
workflow implement codex
# Or:
workflow implement claude
```

Studio passes the current specification and repository rules automatically. The assistant works synchronously in the project, reports its own checks, and returns to the same Studio prompt. No request path has to be copied and there is no background assistant job. On later changes, the shortcut `workflow refine "CHANGE" --implement` saves the refinement and starts the configured assistant; adding `--review` enters guided review on return. These options sequence existing steps and never accept a result.

After it returns, enter:

```
workflow status
```

The expected state is *awaiting human review*. *Assistant checks* are reported separately as passed, failed, or incomplete. This is evidence reported by the assistant, whereas `workflow validate` is ZipperGen checking the current code. Do not accept a result with unresolved failures.

If no coding assistant is installed

The prompt-to-code step necessarily needs a repository-aware coding assistant, but the rest of ZipperGen does not. Leave Studio with `exit`, then select the repository's known workflow directly:

```
cd "$HOME/Projects/zippergen-tutorial"
zippergen studio zippergen/examples/tutorial_review.py:tutorial_review
```

Continue with Section 5. The prompt-generated workflow and the shipped example have similar review-and-retry behavior. Skip the implementation-acceptance instruction in Section 8 because the shipped source is not an open assistant result.

5 Discover, select, and inspect the workflow

For assistant-generated source, enter:

```
workflow list
workflow select
```

Studio lists top-level `@workflow` entry points. Select the new reviewed-answer workflow by number. A displayed reference such as `workflows/reviewed_answer.py:reviewed_answer` means *Python file: workflow object*; it is not a deployment name.

Now inspect the authored source and the semantic protocol:

```
workflow files
workflow show source
workflow show protocol
workflow show communications
workflow show agent Writer
workflow show agent Reviewer
workflow show full
```

`workflow files` reveals supporting modules and declared resources. The protocol view shows the global behavior; communications hides local actions; an agent view is that participant's exact projection; the full view includes actions, prompts, and deployment metadata.

Check five things before continuing:

1. Requester initially owns both inputs.
2. Writer and Reviewer communicate explicitly.
3. The participant that knows the retry condition owns that decision.
4. Every revised draft returns to review and human approval.
5. No unapproved draft can reach Requester.

6 Validate and use the mock model

Inspection answers what the code says. Validation additionally checks the global protocol, ownership, action metadata, deployment declarations, and every generated projection:

```
workflow validate
models default mock
```

```
models assignments
```

A green validation result means the deterministic checks passed; it is not a claim that the prompts are useful or the application intent is correct. The built-in mock model needs no account, network connection, or API key. `models assignments` should list Writer and Reviewer because they contain LLM actions. Its *Last check* column is cached: displaying the table never contacts a provider. Enter `models assignments check` when you want a fresh check of precisely the configurations used by this workflow. If both participants share a configuration, Studio checks it only once.

Validation is not acceptance

`workflow validate` is a repeatable technical check of the Python workflow as it exists now: it checks structure, ownership, declarations, and every local projection. It neither approves the requirements nor closes the current implementation.

`workflow accept` is the later human decision: it records the exact specification and workflow semantics that you reviewed, then closes the implementation task. It does not validate, merge, or repair anything. A workflow may therefore be valid but not yet accepted. Studio displays the two states separately.

`workflow review` is the guided process containing inspection, validation, running, and the explicit acceptance decision. Its presentation is more compact when no human, effect, assistant, or deployment boundary is declared, but acceptance is never inferred or automatic: LLM calls and prompt changes can still be consequential.

If an existing workflow has no open implementation task—for example after upgrading a project accepted by an older Studio—the workflow still exists. `current` says *Implementation task: none* while continuing to show the selected workflow. In that state, `workflow accept` offers a one-time baseline acceptance directly. It records the reviewed specification, semantics, and source snapshot without inventing a refinement and without running or deploying anything.

---

7 Run, interrupt, and resume

Enter:

```
run
```

Before asking for inputs, Studio checks the model configuration used by each participant containing an LLM action. It checks a shared configuration only once, stops before creating a run if an endpoint or model cannot be verified, and reports the configuration that needs attention. An unused default such as `mock` is neither checked nor presented as the run's model. The development-run summary instead lists the effective route for Writer and Reviewer separately.

Studio validates the selected workflow and asks for its declared inputs. Press Enter to accept the default request, enter `3` for `max_retries`, reject the first draft with `n`, and approve the revision with `y`. Mock output such as `[draft_reply:draft]` is expected; the important result is the visible coordination and second review.

Run it once more. At the human approval question press Control-C. Studio should return to its prompt and record the managed run as interrupted. Enter:

```
current
resume
```

The same pending decision reappears. Studio reuses the recorded inputs, model routing, and unique SQLite store. To prove that this is not merely in-memory state, interrupt again, leave Studio, open a new terminal, and run:

```
cd "$HOME/Projects/zippergen-tutorial"
zippergen
current
resume
```

No export or store path needs to be reconstructed after the terminal closes or the computer restarts.

See where every participant is

After an interruption, enter at the Studio prompt:

```
run inspect
run inspect HumanApprover
```

To inspect without interrupting an active foreground run, open a second terminal in the project root and enter:

```
zippergen studio --command "run inspect HumanApprover"
```

The foreground terminal continues to own its input and human-approval prompt; the second command only reads the durable observation state. The first command shows one bounded row per participant: running, waiting to receive, waiting for a human, executing an external action, completed, or failed. It automatically focuses the participant most likely to need attention. The second command selects one participant explicitly and renders its exact local projection with a pointer marking the current durable program position. A local parallel program can have several pointers.

This is observation state, not the recovery snapshot and not an ever-growing trace. Studio does not display workflow variables, action inputs, provider keys, or other environment values in this view.

Inspect the durable store without copying its path

Studio remembers the SQLite store behind the run. Enter:

```
store list
store show
store tasks
store approve
store trace
```

`store list` distinguishes development-run, deployment, standalone, missing, waiting, and completed stores. `store use NUMBER` selects one when several exist; the short `show`, `tasks`, and `approve`, and `trace` forms then use that selection. With one pending task, bare `store approve` selects it automatically; with several, Studio opens a numbered selector. Before accepting any answer, Studio shows the stored instruction and complete rendered context—for this example, the draft and automated reviewer notes—together with the meaning of yes and no. `store tasks` shows the same evidence while listing pending decisions. Stores are normally created by `run` or `deploy`; beginners do not need `store create`. Deletion is recoverable and refuses stores used by an active deployment.

8 Review and accept assistant-generated work

If a coding assistant created the workflow, enter:

```
workflow review
```

The guided review offers the specification, source, semantic views, validation, a development run, and final acceptance. Before the menu, Studio shows the exact specification changes and the semantic workflow changes relative to the pre-change baseline. The same comparison is available directly with:

```
workflow diff
```

Choose *Accept reviewed implementation* only after the visible code and checks are satisfactory. Acceptance records the human decision; it does not delete the specification, source, tests, run history, or private assistant history. It also preserves a private, content-hashed snapshot of the accepted workflow files and records the Git commit and reviewed-file dirty state when Git is available. It does not run or deploy anything. If the specification or workflow semantics later change, `current`, `workflow status`, `workflow validate`, `run`, and `deploy` report that the current files no longer match that accepted baseline. Development runs may still exercise the candidate. Deployment stops and offers the immutable accepted version, return to review, an audited unreviewed override, or cancellation.

In an ordinary terminal, inspect the versioned result:

```
cd "$HOME/Projects/zippergen-tutorial"
git status --short
git diff --check
git diff
```

Do not commit secrets. A commit is a useful local recovery point, but pushing is a separate decision and is not required for local deployment.

9 Prepare a named deployment

Return to Studio. Keep the mock model selected and prepare the deployment without starting a background service:

```
models default mock
deploy reviewed-answer-tutorial --no-start
deployment doctor
deployment show
```

`deploy` uses the immutable accepted source snapshot by default, validates it again, collects declared inputs, creates a separate timestamped deployment bundle and managed Python environment, and remembers the deployment name. The `--no-start` flag deliberately leaves the service stopped. `deployment doctor` runs detailed readiness checks. `deployment show` separates the immutable bundle, supervised service, workflow run, and SQLite store. Warnings about a missing first-run log or store and an uninstalled service are expected before the first start; a red failure must be investigated. A selected API model whose private provider key was not copied into deployment storage is now a blocking failure, not a service that is allowed to enter a restart loop. If a local model is selected, Studio also copies its configured OpenAI-compatible endpoint into the deployment profile automatically. Studio shows a concise readiness summary during `deploy`; `deployment doctor` expands every individual check when needed. Explicit `deployment start`

and `deployment restart` repeat readiness checks before changing service-manager state. Failures block the operation instead of creating a restart loop; warnings do not.

If the working tree changed after acceptance, Studio first shows the exact specification and semantic differences. Choose the accepted snapshot or return to `workflow review`. The exceptional `deploy NAME --unreviewed --reason "TEXT"` form deploys the current candidate and records the reason; it is not the normal tutorial path. A running service continues to use its own bundle and cannot see later working-tree edits.

> **Starting is an operational action**
>
> Continue only on your own development machine, with the mock model, when starting a user service is intentional. A real provider can cost money and a workflow effect can change an external system.

When the readiness checks are acceptable:

```
deployment start
deployment show
deployment inspect
deployment inspect HumanApprover
deployment tasks
deployment approve
deployment logs
deployment stop
```

The finite workflow should reach a waiting human task. Each named deployment owns one stable logical store, so normal operation never requires selecting or copying a store. Bare `deployment inspect` uses the remembered deployment name; with one argument that is not a deployment name, Studio treats it as the participant to focus. It reads the durable positions from the deployment store, so no debugger connection is required. `deployment tasks` displays the proposed draft, reviewer notes, instruction, and decision labels. `deployment approve` repeats that evidence immediately before asking for the decision; an unseen draft is never presented for blind approval. `store` remains available for development runs and advanced recovery.

The tutorial workflow also declares an optional logical Telegram approval connector. It is not required for the first run. To deliver the same durable decision to a private Telegram chat, configure and bind it before the next deployment:

```
deployment connectors setup telegram
deployment connectors bind human-approval telegram-approvals
deploy
deployment notify
```

The bot token is stored privately and is never written to the workflow or ordinary deployment profile. `deployment notify` sends pending decisions and collects replies in one pass; run it again after replying in Telegram. `deployment tasks` always shows the same underlying decision and remains the local inspection path. `deployment stop` prevents the tutorial service from being supervised indefinitely; it preserves the deployment profile, bundle, log, and durable store. If no supported macOS or Linux user service manager is available, the prepared deployment is still valid. Stop at the reported error and use the manual's foreground-deployment section later. The generated service retries failures but does not restart a finite workflow after it completes successfully.

10 What you have established

You have now completed the essential ZipperGen cycle:

- durable intent in `specification.md`;
- visible Python proposed by an assistant, or known shipped source;
- global and participant-level inspection;
- deterministic validation of every projection;
- a free mock execution with explicit human authority;
- interruption and recovery from the same SQLite history;
- explicit human review of generated work; and
- a named, diagnosed deployment prepared before service startup.

The companion `docs/workflow-development-deployment-guide.tex` is the complete manual. Use it next for real provider connections and reusable model configurations, refinement, semantic diffs, external approval adapters, secrets, foreground deployment, logs, restart, recovery, troubleshooting, and the complete command sheet. Build this tutorial alone with `make docs-first-workflow`, or both documents with `make docs`; `docs/README.md` lists the TeX requirements.